Синтаксичний аналіз методом рекурсивного спуску

Мета роботи — програмна реалізація синтаксичного аналізатора (парсера) мови Krok методом рекурсивного спуску.

1. Коротка характеристика мови

Krok — проста імперативна процедурна мова програмування загального призначення, статично і суворо типізована; мова-прототип — Go, рівень складності 2. Програма складається з одного файлу: `package main`, `import "fmt"`, оголошення змінних, констант, типів і функцій. Мова має типи `int`, `float`, `bool`, `string`, користувацькі скалярні типи, масиви та структури; інструкції присвоєння, розгалуження `if / else if / else`, циклу `for` (з передумовою та трикомпонентного), введення й виведення через пакет `fmt`. Крапку з комою наприкінці рядка вставляє лексичний аналізатор.

2. Повна граматика мови

Граматика належить до класу LL(1) і не містить лівої рекурсії: ліва асоціативність бінарних операторів виражена повтореннями `{ ... }`, права асоціативність `**` — правою рекурсією через `UnaryExpr`. Тому кожне правило безпосередньо перетворюється на функцію рекурсивного спуску. Синтаксична частина граматики (аксіома `Program`):

```
Program        = PackageClause ';' { ImportDecl ';' } { TopLevelDecl ';' }
PackageClause  = package Ident
ImportDecl     = import StringLit
TopLevelDecl   = VarDecl | ConstDecl | TypeDecl | FuncDecl

(* § Оголошення *)
VarDecl        = var IdentList
                 ( Type [ '=' ExpressionList ] | '=' ExpressionList )
ConstDecl      = const Ident [ Type ] '=' Expression
TypeDecl       = type Ident ( Type | StructType )
StructType     = struct '{' FieldList '}'
FieldList      = [ FieldDecl FieldTail ]
FieldTail      = [ ';' FieldList ]
FieldDecl      = IdentList Type
IdentList      = Ident { ',' Ident }

(* § Типи *)
Type           = BasicType | TypeName | ArrayType
BasicType      = int | float | bool | string
TypeName       = Ident
ArrayType      = '[' Expression ']' Type

(* § Функції *)
FuncDecl       = func Ident '(' [ ParamList ] ')' [ Type ] Block
ParamList      = ParamGroup { ',' ParamGroup }
ParamGroup     = IdentList ParamType
ParamType      = [ '*' ] Type

(* § Інструкції *)
Block          = '{' StatementList '}'
StatementList  = [ Statement StatementTail ]
StatementTail  = [ ';' StatementList ]
Statement      = VarDecl | ConstDecl | IfStmt | ForStmt | ReturnStmt
               | IoStmt | Block | SimpleStmt
SimpleStmt     = Expression [ AssignOp Expression | IncDecOp ]
AssignOp       = '=' | ':='
IncDecOp       = '++' | '--'
IfStmt         = if Expression Block [ else ( IfStmt | Block ) ]
ForStmt        = for ForHeader Block
```

```
ForHeader      = SimpleStmt [ ';' Expression ';' SimpleStmt ]
ReturnStmt     = return [ Expression ]
IoStmt         = fmt '.' ( OutputCall | InputCall )
OutputCall     = ( 'Print' | 'Println' ) '(' [ ExpressionList ] ')'
InputCall      = 'Scan' '(' AddrExpr { ',' AddrExpr } ')'

(* § Вирази *)
ExpressionList = Expression { ',' Expression }
Expression     = OrExpr
OrExpr         = AndExpr { '||' AndExpr }
AndExpr        = RelExpr { '&&' RelExpr }
RelExpr        = AddExpr [ RelOp AddExpr ]
AddExpr        = MulExpr { AddOp MulExpr }
MulExpr        = UnaryExpr { MulOp UnaryExpr }
UnaryExpr      = UnaryOp UnaryExpr | PowerExpr
PowerExpr      = PrimaryExpr [ '**' UnaryExpr ]
PrimaryExpr    = Operand { Selector | Index | Arguments }
Selector       = '.' Ident
Index          = '[' Expression ']'
Arguments      = '(' [ ArgumentList ] ')'
ArgumentList   = Argument { ',' Argument }
Argument       = AddrExpr | Expression
AddrExpr       = '&' PrimaryExpr
Operand        = Literal | Ident | Dereference | Conversion | LenCall
               | ArrayLit | '(' Expression ')'
Dereference    = '*' Ident
Conversion     = BasicType '(' Expression ')'
LenCall        = len '(' Expression ')'
ArrayLit       = ArrayType LiteralValue
LiteralValue   = '{' [ Element ElementTail ] '}'
ElementTail    = [ ',' [ Element ElementTail ] ]
Element        = Expression | LiteralValue
Literal        = IntLit | FloatLit | StringLit | BoolLit
BoolLit        = true | false
RelOp          = '==' | '!=' | '<' | '<=' | '>' | '>='
AddOp          = '+' | '-'
MulOp          = '*' | '/' | '%'
UnaryOp        = '-' | '+' | '!'
```

Лексична частина граматики (`Ident`, `IntLit`, `FloatLit`, `StringLit` та ін.) реалізована лексичним аналізатором КП 1; для парсера ці нетермінали є токенами `id`, `intnum`, `floatnum`, `strlit`, `boolval`.

3. Лексичний аналізатор

Використано лексичний аналізатор `krok_lex.py` з роботи КП 1, побудований методом діаграми станів. Парсер викликає функцію `analyze()` і отримує таблицю розбору `tableOfSymb` у форматі `{ n_rec : (num_line, lexeme, token, idxIdConst) }`. Синтаксичний аналіз виконується лише тоді, коли лексичний аналіз завершився успішно. Крапки з комою, вставлені лексичним аналізатором наприкінці рядків, потрапляють у таблицю розбору як звичайні токени (`;, punct`).

4. Програмна реалізація синтаксичного аналізатора

Парсер `krok_parse.py` побудовано за зразком `parse_02.py`. Глобальна змінна `numRow` — номер поточного запису таблиці розбору; `getSymb()` повертає номер рядка, лексему й токен поточного запису; `parseToken(lexeme, token)` перевіряє, що поточний запис збігається з очікуваним, і переходить до наступного; `failParse(str, tuple)` виводить діагностичне повідомлення і аварійно завершує роботу.

Нетерміналам синтаксичної граматики відповідають функції `parse<Нетермінал>()` — усього 56. Окремих функцій не мають лише правила, що складаються з одного токена (`RelOp`, `AddOp`, `MulOp`, `UnaryOp`, `AssignOp`, `IncDecOp`, `BoolLit`, `TypeName = Ident`): їх перевіряє безпосередньо порівняння поточного токена. Альтернативу обирає поточна лексема (граматика LL(1)), повторення `{ ... }` реалізовано циклом `while`, необов'язкову частину `[ ... ]` — умовою `if`.

Відмінності від зразка: (1) функція `peek()` повертає поточну лексему й токен без аварійного завершення в кінці таблиці — вона потрібна для вибору альтернатив і повторень; (2) `parseToken(None, token)` приймає будь-яку лексему заданого токена (ідентифікатор, літерал), а `expect(lexeme)` бере токен з таблиці лексем мови; (3) відступи у трасі розбору й виведення імені функції виконує декоратор `@rule`, а не явні виклики `nextIndt()` і `predIndt()` у кожній функції.

Коди аварійного завершення: 1 — невідповідність токенів; 2 — невідповідність інструкцій; 3 — помилка у виразі; 4 — помилка у типі; 5 — невідповідність оголошень верхнього рівня; 1001, 1002 — неочікуваний кінець програми.

5. План тестування

Таблиця 1 — План тестування

№	Тип випробування	Очікуваний результат	Кількість
1	пропуск термінала	помилка, локалізація (номер рядка), діагностичне повідомлення	3
2	зайвий термінал	помилка, локалізація (номер рядка), діагностичне повідомлення	3
3	ключове слово	помилка, локалізація (номер рядка), діагностичне повідомлення	2
4	помилковий оператор	помилка, локалізація (номер рядка), діагностичне повідомлення	3
5	пріоритет	успішний розбір	2
6	асоціативність	успішний розбір	2
7	програми	успішний розбір	10
8	некоректні програми	помилка, локалізація (номер рядка), діагностичне повідомлення	8

Усі випадки виконуються автоматично програмою `tests/run_tests.py`; результат кожного додатково звіряється з незалежним розпізнавачем, що працює безпосередньо за граматикою специфікації мови. Обидва аналізатори ухвалили однакові рішення в усіх 33 випадках.

6. Протокол тестування

Тест № 1 (1 пропуск термінала): немає {.

```
1  package main
2
3  import "fmt"
4
5  func main() {
6  if x > 0
7      y = 1
8  }
9  }
```

```
>python krok_parse.py test.krok
...
                                    parseLiteral():
                                      parseToken: В рядку 6 токен ('0', 'intnum')
               parseBlock():
Parser ERROR:
     В рядку 6 неочікуваний елемент (;,punct).
     Очікувався - ({,brace).
Parser: Аварійне завершення програми з кодом 1
```

Оцінка: відсутній термінал виявлено в тому місці, де він очікувався; повідомлення вказує номер рядка, знайдену й очікувану лексеми.

Тест № 2 (1 пропуск термінала): немає).

```
 1  package main
 2
 3  import "fmt"
 4
 5  func main() {
 6  fmt.Println(x
 7  }
```

```
>python krok_parse.py test.krok
...
                                    parsePrimaryExpr():
                                      parseOperand():
                                        parseToken: В рядку 6 токен ('x', 'id')
Parser ERROR:
     В рядку 6 неочікуваний елемент (;,punct).
     Очікувався - (),paren).
Parser: Аварійне завершення програми з кодом 1
```

Оцінка: відсутній термінал виявлено в тому місці, де він очікувався; повідомлення вказує номер рядка, знайдену й очікувану лексеми.

Тест № 3 (1 пропуск термінала): немає ; у заголовку for.

```
 1  package main
 2
 3  import "fmt"
 4
 5  func main() {
 6  for i := 0; i < 3 i++ {
 7  }
 8  }
```

```
>python krok_parse.py test.krok
...
                                    parseOperand():
                                      parseLiteral():
                                        parseToken: В рядку 6 токен ('3', 'intnum')
Parser ERROR:
     В рядку 6 неочікуваний елемент (i,id).
     Очікувався - (;,punct).
Parser: Аварійне завершення програми з кодом 1
```

Оцінка: відсутній термінал виявлено в тому місці, де він очікувався; повідомлення вказує номер рядка, знайдену й очікувану лексеми.

Тест № 4 (2 зайвий термінал): зайва).

```
 1  package main
 2
 3  import "fmt"
 4
 5  func main() {
 6  x = (1 + 2))
```

```
 7  }
```

```
>python krok_parse.py test.krok
...
                                                                parseToken: В рядку 6 токен ('2',
    'intnum')
                                    parseToken: В рядку 6 токен (')', 'paren')
           parseStatementTail():
Parser ERROR:
     В рядку 6 неочікуваний елемент (),paren).
     Очікувався - (},brace).
Parser: Аварійне завершення програми з кодом 1
```

Оцінка: зайва лексема не відповідає жодному правилу на цій позиції — розбір аварійно завершено.

Тест № 5 (2 зайвий термінал): зайва кома.

```
 1  package main
 2
 3  import "fmt"
 4
 5  func main() {
 6  fmt.Println(x,)
 7  }
```

```
>python krok_parse.py test.krok
...
                                  parsePowerExpr():
                                    parsePrimaryExpr():
                                      parseOperand():
Parser ERROR:
     В рядку 6 неочікуваний елемент (),paren).
     Очікувався - операнд.
Parser: Аварійне завершення програми з кодом 3
```

Оцінка: зайва лексема не відповідає жодному правилу на цій позиції — розбір аварійно завершено.

Тест № 6 (2 зайвий термінал): зайве int.

```
 1  package main
 2  var x int int
```

```
>python krok_parse.py test.krok
...
        parseType():
          parseBasicType():
            parseToken: В рядку 2 токен ('int', 'keyword')
Parser ERROR:
     В рядку 2 неочікуваний елемент (int,keyword).
     Очікувався - (;,punct).
Parser: Аварійне завершення програми з кодом 1
```

Оцінка: зайва лексема не відповідає жодному правилу на цій позиції — розбір аварійно завершено.

Тест № 7 (3 ключове слово): fucn.

```
 1  package main
 2  fucn main() {
 3  }
```

```
>python krok_parse.py test.krok
...
      parseToken: В рядку 1 токен ('main', 'id')
```

```
    parseToken: В рядку 1 токен (';', 'punct')
    parseTopLevelDecl():
Parser ERROR:
     В рядку 2 неочікуваний елемент (fucn,id).
     Очікувався - var, const, type або func.
Parser: Аварійне завершення програми з кодом 5
```

Оцінка: лексичний аналізатор розпізнав помилково написане ключове слово як ідентифікатор, а парсер виявив, що на цьому місці ідентифікатор неприпустимий.

Тест № 8 (3 ключове слово): retrun.

```
 1  package main
 2
 3  import "fmt"
 4
 5  func main() {
 6  retrun 1
 7  }
```

```
>python krok_parse.py test.krok
...
                                  parseOperand():
                                    parseToken: В рядку 6 токен ('retrun', 'id')
           parseStatementTail():
Parser ERROR:
     В рядку 6 неочікуваний елемент (1,intnum).
     Очікувався - (},brace).
Parser: Аварійне завершення програми з кодом 1
```

Оцінка: лексичний аналізатор розпізнав помилково написане ключове слово як ідентифікатор, а парсер виявив, що на цьому місці ідентифікатор неприпустимий.

Тест № 9 (4 помилковий оператор): + *.

```
 1  package main
 2
 3  import "fmt"
 4
 5  func main() {
 6  x = 3 + * 4
 7  }
```

```
>python krok_parse.py test.krok
...
                                  parseOperand():
                                    parseDereference():
                                      parseToken: В рядку 6 токен ('*', 'mult_op')
Parser ERROR:
     В рядку 6 неочікуваний елемент (4,intnum).
     Очікувався - (id,id).
Parser: Аварійне завершення програми з кодом 1
```

Оцінка: недопустиму послідовність операторів виявлено; `a < b < c` відхилено, бо оператори порівняння неасоціативні.

Тест № 10 (4 помилковий оператор): <>.

```
 1  package main
 2
 3  import "fmt"
 4
 5  func main() {
 6  a = b <> c
 7  }
```

```
>python krok_parse.py test.krok
```

```
...
                              parsePowerExpr():
                                parsePrimaryExpr():
                                  parseOperand():
Parser ERROR:
     В рядку 6 неочікуваний елемент (>,rel_op).
     Очікувався - операнд.
Parser: Аварійне завершення програми з кодом 3
```

Оцінка: недопустиму послідовність операторів виявлено; `a < b < c` відхилено, бо оператори порівняння неасоціативні.

Тест № 11 (4 помилковий оператор): a < b < c.

```
 1  package main
 2
 3  import "fmt"
 4
 5  func main() {
 6  x = 1 < 2 < 3
 7  }
```

```
>python krok_parse.py test.krok
...
                                    parseLiteral():
                                      parseToken: В рядку 6 токен ('2', 'intnum')
            parseStatementTail():
Parser ERROR:
     В рядку 6 неочікуваний елемент (<,rel_op).
     Очікувався - (},brace).
Parser: Аварійне завершення програми з кодом 1
```

Оцінка: недопустиму послідовність операторів виявлено; `a < b < c` відхилено, бо оператори порівняння неасоціативні.

Тест № 12 (5 пріоритет): a + b * c.

```
 1  package main
 2
 3  import "fmt"
 4
 5  func main() {
 6  x = a + b * c
 7  }
```

```
>python krok_parse.py test.krok
...
              parseSimpleStmt():
                parseExpression():
                  parseOrExpr():
                    parseAndExpr():
                      parseRelExpr():
                        parseAddExpr():
                          parseMulExpr():
                            parseUnaryExpr():
                              parsePowerExpr():
                                parsePrimaryExpr():
                                  parseOperand():
                                    parseToken: В рядку 6 токен ('x', 'id')
                parseToken: В рядку 6 токен ('=', 'assign_op')
                parseExpression():
                  parseOrExpr():
                    parseAndExpr():
                      parseRelExpr():
                        parseAddExpr():
                          parseMulExpr():
                            parseUnaryExpr():
                              parsePowerExpr():
                                parsePrimaryExpr():
```

```
                                parseOperand():
                                  parseToken: В рядку 6 токен ('a', 'id')
                            parseToken: В рядку 6 токен ('+', 'add_op')
                            parseMulExpr():
                              parseUnaryExpr():
                                parsePowerExpr():
                                  parsePrimaryExpr():
                                    parseOperand():
                                      parseToken: В рядку 6 токен ('b', 'id')
                              parseToken: В рядку 6 токен ('*', 'mult_op')
                              parseUnaryExpr():
                                parsePowerExpr():
                                  parsePrimaryExpr():
                                    parseOperand():
                                      parseToken: В рядку 6 токен ('c', 'id')
...
Parser: Синтаксичний аналіз завершився успішно
```

Оцінка: у трасі розбору множення вкладене глибше за додавання (`parseMulExpr` усередині `parseAddExpr`), тобто виконується раніше — пріоритет операторів дотримано.

Тест № 13 (5 пріоритет): -2 ** 2 && !f || g.

```
1  package main
2
3  import "fmt"
4
5  func main() {
6  ok = -2 ** 2 > 0 && !f || g
7  }
```

```
>python krok_parse.py test.krok
...
              parseSimpleStmt():
                parseExpression():
                  parseOrExpr():
                    parseAndExpr():
                      parseRelExpr():
                        parseAddExpr():
                          parseMulExpr():
                            parseUnaryExpr():
                              parsePowerExpr():
                                parsePrimaryExpr():
                                  parseOperand():
                                    parseToken: В рядку 6 токен ('ok', 'id')
                parseToken: В рядку 6 токен ('=', 'assign_op')
                parseExpression():
                  parseOrExpr():
                    parseAndExpr():
                      parseRelExpr():
                        parseAddExpr():
                          parseMulExpr():
                            parseUnaryExpr():
                              parseToken: В рядку 6 токен ('-', 'add_op')
                              parseUnaryExpr():
                                parsePowerExpr():
                                  parsePrimaryExpr():
                                    parseOperand():
                                      parseLiteral():
                                        parseToken: В рядку 6 токен ('2', 'intnum')
                                  parseToken: В рядку 6 токен ('**', 'pow_op')
                                  parseUnaryExpr():
                                    parsePowerExpr():
                                      parsePrimaryExpr():
                                        parseOperand():
                                          parseLiteral():
                                            parseToken: В рядку 6 токен ('2', 'intnum')
                        parseToken: В рядку 6 токен ('>', 'rel_op')
                        parseAddExpr():
                          parseMulExpr():
```

```
                          parseUnaryExpr():
                            parsePowerExpr():
                              parsePrimaryExpr():
                                parseOperand():
                                  parseLiteral():
                                    parseToken: В рядку 6 токен ('0', 'intnum')
                      parseToken: В рядку 6 токен ('&&', 'and_op')
                      parseRelExpr():
                        parseAddExpr():
                          parseMulExpr():
                            parseUnaryExpr():
                              parseToken: В рядку 6 токен ('!', 'not_op')
                              parseUnaryExpr():
                                parsePowerExpr():
                                  parsePrimaryExpr():
                                    parseOperand():
                                      parseToken: В рядку 6 токен ('f', 'id')
                    parseToken: В рядку 6 токен ('||', 'or_op')
                    parseAndExpr():
                      parseRelExpr():
                        parseAddExpr():
                          parseMulExpr():
                            parseUnaryExpr():
                              parsePowerExpr():
                                parsePrimaryExpr():
                                  parseOperand():
                                    parseToken: В рядку 6 токен ('g', 'id')
...
Parser: Синтаксичний аналіз завершився успішно
```

Оцінка: у трасі розбору множення вкладене глибше за додавання (`parseMulExpr` усередині `parseAddExpr`), тобто виконується раніше — пріоритет операторів дотримано.

Тест № 14 (6 асоціативність): a - b - c.

```
1  package main
2
3  import "fmt"
4
5  func main() {
6  x = a - b - c
7  }
```

```
>python krok_parse.py test.krok
...
              parseSimpleStmt():
                parseExpression():
                  parseOrExpr():
                    parseAndExpr():
                      parseRelExpr():
                        parseAddExpr():
                          parseMulExpr():
                            parseUnaryExpr():
                              parsePowerExpr():
                                parsePrimaryExpr():
                                  parseOperand():
                                    parseToken: В рядку 6 токен ('x', 'id')
                parseToken: В рядку 6 токен ('=', 'assign_op')
                parseExpression():
                  parseOrExpr():
                    parseAndExpr():
                      parseRelExpr():
                        parseAddExpr():
                          parseMulExpr():
                            parseUnaryExpr():
                              parsePowerExpr():
                                parsePrimaryExpr():
                                  parseOperand():
                                    parseToken: В рядку 6 токен ('a', 'id')
                          parseToken: В рядку 6 токен ('-', 'add_op')
```

```
                          parseMulExpr():
                            parseUnaryExpr():
                              parsePowerExpr():
                                parsePrimaryExpr():
                                  parseOperand():
                                    parseToken: В рядку 6 токен ('b', 'id')
                          parseToken: В рядку 6 токен ('-', 'add_op')
                          parseMulExpr():
                            parseUnaryExpr():
                              parsePowerExpr():
                                parsePrimaryExpr():
                                  parseOperand():
                                    parseToken: В рядку 6 токен ('c', 'id')
...
Parser: Синтаксичний аналіз завершився успішно
```

Оцінка: для `a - b - c` обидва віднімання розібрано циклом одного виклику `parseAddExpr` (ліва асоціативність); для `2 ** 3 ** 2` другий `**` вкладено у праву частину першого (права асоціативність).

Тест № 15 (6 асоціативність): 2 ** 3 ** 2.

```
1  package main
2
3  import "fmt"
4
5  func main() {
6  y = 2 ** 3 ** 2
7  }
```

```
>python krok_parse.py test.krok
...
              parseSimpleStmt():
                parseExpression():
                  parseOrExpr():
                    parseAndExpr():
                      parseRelExpr():
                        parseAddExpr():
                          parseMulExpr():
                            parseUnaryExpr():
                              parsePowerExpr():
                                parsePrimaryExpr():
                                  parseOperand():
                                    parseToken: В рядку 6 токен ('y', 'id')
                parseToken: В рядку 6 токен ('=', 'assign_op')
                parseExpression():
                  parseOrExpr():
                    parseAndExpr():
                      parseRelExpr():
                        parseAddExpr():
                          parseMulExpr():
                            parseUnaryExpr():
                              parsePowerExpr():
                                parsePrimaryExpr():
                                  parseOperand():
                                    parseLiteral():
                                      parseToken: В рядку 6 токен ('2', 'intnum')
                                parseToken: В рядку 6 токен ('**', 'pow_op')
                                parseUnaryExpr():
                                  parsePowerExpr():
                                    parsePrimaryExpr():
                                      parseOperand():
                                        parseLiteral():
                                          parseToken: В рядку 6 токен ('3', 'intnum')
                                    parseToken: В рядку 6 токен ('**', 'pow_op')
                                    parseUnaryExpr():
                                      parsePowerExpr():
                                        parsePrimaryExpr():
                                          parseOperand():
                                            parseLiteral():
```

```
                                                parseToken: В рядку 6 токен ('2', 'intnum')
...
Parser: Синтаксичний аналіз завершився успішно
```

Оцінка: для `a - b - c` обидва віднімання розібрано циклом одного виклику `parseAddExpr` (ліва асоціативність); для `2 ** 3 ** 2` другий `**` вкладено у праву частину першого (права асоціативність).

Тест № 16 (7 програми): nested.krok.

```
>python krok_parse.py test.krok
Lexer: Лексичний аналіз завершено успішно
------------------------------
('len_tableOfSymb', 137)
... (606 рядків пропущено) ...
                  parseToken: В рядку 17 токен (';', 'punct')
                  parseStatementList():
          parseToken: В рядку 18 токен ('}', 'brace')
    parseToken: В рядку 18 токен (';', 'punct')
Parser: Синтаксичний аналіз завершився успішно
```

Оцінка: вкладені цикли, розгалуження й літерали масивів розібрано успішно; глибина вкладеності видна з відступів у трасі.

7. Висновки

Реалізовано синтаксичний аналізатор мови Krok методом рекурсивного спуску: 56 функцій розбору за правилами синтаксичної граматики. Аналізатор працює з таблицею розбору лексичного аналізатора КП 1, розбирає всі конструкції мови й локалізує синтаксичні помилки з точністю до рядка.

Завдяки тому, що граматику ще на етапі специфікації було приведено до класу LL(1) і звільнено від лівої рекурсії, жодних додаткових трансформацій не знадобилося: для вибору альтернативи завжди достатньо однієї поточної лексеми, а розбір відбувається без повернень.

Найцікавішим виявився розбір простої інструкції: присвоєння, оголошення з ініціалізацією, інкремент і виклик функції починаються з виразу, тож парсер спершу розбирає вираз і лише потім, за наступною лексемою, з'ясовує, що це за інструкція. Перевірку того, чи є ліва частина присвоєння l-значенням, відкладено до семантичного аналізу (КП 3).

Коректність підтверджено 33 тестами за планом із табл. 1 та звіркою з незалежним розпізнавачем, що працює безпосередньо за граматикою специфікації: рішення обох аналізаторів збіглися в усіх випадках.